

# BÁO CÁO THỰC HÀNH

**Thực hành Nhập môn Mạng máy tính**

**Lab 2: Mật mã học hiện đại**

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 30/03/2026

## Nhóm 13 – IT005.Q22.1

### THÔNG TIN CHUNG

| MSSV     | Họ và tên           | Nội dung báo cáo | % công việc |
|----------|---------------------|------------------|-------------|
| 24521388 | Nguyễn Hoàng Phúc   | Câu 4            | 50%         |
| 23520536 | Phạm Cao Minh Hoàng | Câu 5            | 50%         |

## 1. Nhiệm vụ 2.4

**Nhiệm vụ 2.4**

SEED Lab - Secret-Key Encryption Lab (Task 5):

Sinh viên thực hiện các bước sau bằng Python để hiểu rõ về đặc tính lan truyền lỗi (error propagation) của các chế độ mã hóa khác nhau:

1. Tạo dữ liệu 1000 byte.
2. Mã hóa dữ liệu bằng AES-128.
3. Làm hỏng bản mã bằng cách đảo 1 bit tại byte thứ 26.
4. Giải mã bản mã đã bị lỗi và *quan sát có bao nhiêu khối dữ liệu bản rõ bị hỏng với chế độ mã hóa* được sử dụng lần lượt là ECB, CBC, CFB, hoặc OFB.

**Mã nguồn:**

```
import os
from Crypto.Cipher import AES
from Crypto.Util import Counter
from Crypto.Util.Padding import pad

key = b'1234567890123456'
data = os.urandom(1000)
padded_data = pad(data, 16)

def test_error_propagation(mode_name):
    print(f"\nChế độ: {mode_name}")

    if mode_name == 'ECB':
        cipher = AES.new(key, AES.MODE_ECB)
        ct = cipher.encrypt(padded_data)
        iv = None
        decipher = AES.new(key, AES.MODE_ECB)

    elif mode_name == 'CBC':
        iv = os.urandom(16)
        cipher = AES.new(key, AES.MODE_CBC, iv)
        ct = cipher.encrypt(padded_data)
        decipher = AES.new(key, AES.MODE_CBC, iv)

    elif mode_name == 'CFB':
        iv = os.urandom(16)
        cipher = AES.new(key, AES.MODE_CFB, iv)
```

```

ct = cipher.encrypt(padded_data)
decipher = AES.new(key, AES.MODE_CFB, iv)

elif mode_name == 'OFB':
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_OFB, iv)
    ct = cipher.encrypt(padded_data)
    decipher = AES.new(key, AES.MODE_OFB, iv)

elif mode_name == 'CTR':
    ctr = Counter.new(128, initial_value=0)
    cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
    ct = cipher.encrypt(padded_data)
    ctr2 = Counter.new(128, initial_value=0)
    decipher = AES.new(key, AES.MODE_CTR, counter=ctr2)
    iv = None

ct_list = bytearray(ct)
ct_list[25] ^= 0x01
ct_corrupted = bytes(ct_list)

pt_decrypted = decipher.decrypt(ct_corrupted)
pt_decrypted = pt_decrypted[:1000]

diff_bytes = sum(a != b for a, b in zip(data, pt_decrypted))
print(f"Số byte bản rõ bị hỏng: {diff_bytes}/1000 bytes")
return diff_bytes

modes = ['ECB', 'CBC', 'CFB', 'OFB', 'CTR']
for m in modes:
    test_error_propagation(m)

```

**Các hàm thành phần chính:**

- Hàm pad(): Dùng để tự động đệm dữ liệu vào bội số của 16 byte để phù hợp với kích thước khối của AES.
- Hàm test\_error\_propagation(mode\_name): Hàm trung tâm, thực hiện toàn bộ quá trình mã hóa, sửa lỗi 1 bit trên bản mã, giải mã và tính số byte bị hỏng cho từng chế độ.
- Sử dụng các lớp AES.new() với các chế độ khác nhau (ECB, CBC, CFB, OFB, CTR) từ thư viện pycryptodome.

**Các bước thực hiện:**

- Tạo dữ liệu ngẫu nhiên dài 1000 byte bằng os.urandom().
- Sử dụng hàm pad() để đệm dữ liệu cho phù hợp với block size 16 byte.
- Mã hóa dữ liệu bằng AES-128 với từng chế độ mã hóa (ECB, CBC, CFB, OFB, CTR).
- Làm hỏng bản mã bằng cách đảo ngược 1 bit tại byte thứ 26 (index 25).

- Giải mã bản mã đã bị hỏng và so sánh với dữ liệu gốc để đếm số byte bản rõ bị ảnh hưởng.

### Kết quả thực thi:

```
/Users/hoangphamcaominh/PycharmProjects/PythonProject4/.venv/bin/python /Users/hoangphamcaominh/PycharmProjects/PythonProject4/Lab02_2.4.py

Chế độ: ECB
Số byte bản rõ bị hỏng: 16/1000 bytes

Chế độ: CBC
Số byte bản rõ bị hỏng: 17/1000 bytes

Chế độ: CFB
Số byte bản rõ bị hỏng: 17/1000 bytes

Chế độ: OFB
Số byte bản rõ bị hỏng: 1/1000 bytes

Chế độ: CTR
Số byte bản rõ bị hỏng: 1/1000 bytes

Process finished with exit code 0
```

Hình 2.4.1: Kết quả của code

**Nhận xét:** Kết quả cho thấy rõ sự khác biệt về khả năng lan truyền lỗi giữa các chế độ:

- ECB chỉ làm hỏng 16 byte (đúng 1 khối).
- CBC và CFB làm hỏng khoảng 17 byte (lan truyền sang khối kế tiếp).
- OFB và CTR chỉ làm hỏng 1 byte (lan truyền lỗi rất ít).

### Kết luận:

ECB có khả năng lan truyền lỗi yếu nhất, trong khi OFB và CTR rất phù hợp với môi trường truyền có nhiễu. CBC thể hiện sự cân bằng tốt giữa bảo mật và khả năng phát hiện lỗi.

## 2. Nhiệm vụ 2.5

### Nhiệm vụ 2.5

Tìm hiểu và so sánh độ mạnh và sự khác biệt của DES, Triple-DES (3DES), và AES. Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

### Giới thiệu :

- **DES (Data Encryption Standard):** Là thuật toán mật mã khối đối xứng được phát triển bởi IBM vào những năm 1970. DES sử dụng khóa 56 bit và khối dữ liệu 64 bit. Đây từng là tiêu chuẩn mã hóa của chính phủ Mỹ nhưng hiện nay đã lỗi thời.
- **Double-DES (2DES):** Là phiên bản áp dụng DES hai lần liên tiếp với hai khóa khác nhau, nhằm tăng độ dài khóa lên 112 bit. Tuy nhiên, cách làm này không mang lại độ an toàn như mong đợi.
- **Triple-DES (3DES):** Là phiên bản cải tiến của DES, thực hiện mã hóa ba lần (Encrypt-Decrypt-Encrypt) với ba khóa khác nhau. 3DES có độ dài khóa hiệu dụng 168 bit và vẫn được dùng ở một số hệ thống cũ.
- **AES (Advanced Encryption Standard):** Là tiêu chuẩn mật mã khối hiện đại được NIST chọn năm 2001 để thay thế DES. AES hỗ trợ khóa 128, 192 hoặc 256 bit và khối 128 bit. Đây là thuật toán được sử dụng rộng rãi nhất hiện nay.

### Bảng so sánh:

| Thuật toán | Độ dài khóa         | Kích thước khối | Độ mạnh                      | Nhược điểm chính                  |
|------------|---------------------|-----------------|------------------------------|-----------------------------------|
| DES        | 56 bit              | 64 bit          | Đã bị phá hoàn toàn          | Khóa quá ngắn, dễ bị brute-force  |
| 2DES       | 112 bit             | 64 bit          | Không an toàn                | Dễ bị tấn công Meet-in-the-Middle |
| 3DES       | 168 bit             | 64 bit          | Tạm chấp nhận nhưng lỗi thời | Tốc độ chậm, khối nhỏ             |
| AES        | 128 / 192 / 256 bit | 128 bit         | An toàn cao                  | Không có nhược điểm đáng kể       |

**Kết luận:**

- DES đã hoàn toàn không còn an toàn do khóa chỉ 56 bit.
- Không nên sử dụng 2DES vì dù có khóa 112 bit nhưng thuật toán dễ bị tấn công Meet-in-the-Middle, làm giảm đáng kể độ mạnh thực tế.
- 3DES mạnh hơn DES nhưng tốc độ chậm và vẫn sử dụng khối 64 bit nên chỉ phù hợp với một số hệ thống cũ cần tương thích.
- AES là thuật toán vượt trội nhất nhờ độ dài khóa lớn, khối dữ liệu 128 bit, tốc độ nhanh và độ an toàn cao.

Do đó, trong các ứng dụng hiện đại, nên ưu tiên sử dụng AES-128 hoặc AES-256 để đảm bảo tính bảo mật lâu dài và hiệu suất tốt.

**Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?**

- 2DES không mang lại mức bảo mật tương xứng với độ dài khóa 112 bit.
- Nó dễ bị tấn công Meet-in-the-Middle, khiến việc phá mã trở nên khả thi trên máy tính hiện đại.
- Hiện nay đã có AES (128/192/256 bit) mạnh hơn rất nhiều, tốc độ nhanh hơn và an toàn cao.

**HẾT**